continuously execute a statement so long as the condition remains true. The basic syntax of a `while` statement looks like the following:

```
while (condition) {
  statements
}
```

Using the `while` statement, we could rewrite the previous `for` loop example to blink an LED five times in the following manner:

```
int i = 0;
while (i < 5) {
  digitalWrite(13, HIGH);
  delay(250);
  digitalWrite(13, LOW);
  delay(250);
  i++:
}
```

The first line, `int i = 0;` declares the index variable and assigns the value 0. The program flow reaches the `while` statement and compares the value 5 to the variable `i`, with the first time evaluating as true. The enclosed statements are then executed, which includes the line `i++;` used to increment the value of `i` at the end of the block of code.

When the end of the statements is reached, it loops back to the conditional statement of the `while` loop and if the condition remains true, the following statements are executed all over again. If, however, the condition is false, then the while loop ends and control passes to the following statements in the program.

So with the `for` loop, why would you need the `while` loop? If the previous example was all you were ever going to do with it, then no there is really not much use for it. However, if you think of the `while` statement more as a continuous `if` statement, then it becomes quite useful in the right situation. From our project code, we needed to do one thing over and over again while the switch remained triggered and to prevent the continuation of normal program flow until that condition is no longer met. In other words, as long as the switch was triggered in our project example, we wanted to continue to cycle through each of the seven colors until such time as the switch was no longer triggered.

A simpler way to write this code would be something like the following:

```
while (digitalRead(2) == HIGH) digitalWrite(13, HIGH);
```

In this example, for as long as the digital input on pin 2 is equal to `HIGH`, if for example a switch has been activated and is on, then the `while` loop will continue to keep pin 13 on, or `HIGH`, and will never exit the loop. Only when the conditional expression `digitalRead(2) == HIGH` returns false will the next following line of code be executed.

Do

Like the `for` statement, the `while` statement evaluates its condition before executing its block of code so that if the condition is not met, its code will never run. Sometimes it would be nice to always run the enclosed code at least once before evaluating its condition and for that we would use the `do` statement, or `do` loop. The `do` loop is like a modified `while` statement in that it executes a line or lines of code and then tests for a condition at the end of the loop rather than the beginning before deciding to loop to the beginning again or not. The following is the basic syntax for a `do` loop: